Day 65 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 3 reference

Watch: LSTM Backprop Through Time Formula Derivation (Henry Harvin) | CS231n Winter 2016: Lecture 10 (Andrej Karpathy)

`day65_lstm_backward_pass.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: dh, dc = gradient of the loss with respect to h and c · f, i, g, o = the gates · da = gradient of the four gates' raw scores · $z = [x_t, h_{prev}]$ · m_t = mask

Segment 0 — Setup

Imports and the reused pieces

CODE

```
import numpy as np
from threadpoolctl import threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
```

- `from threadpoolctl import threadpool_limits` — used in main() to keep NumPy's matrix library on one thread: same results, faster here (Day 64).
- `sigmoid, softmax, clip_by_global_norm, make_sign_task` — copied verbatim from Days 57-63, so Day 60's training setup and Day 59's data are reproduced exactly.
- `lstm_step_forward, init_lstm, lstm_forward` — copied verbatim from Days 63 and 64; the cache that lstm_forward returns is today's input.

Segment 1 — The backward pass

lstm_backward

MATH

$$do = dh \cdot \tanh(c), \quad dc_{new} = dc + dh \cdot o \cdot (1 - \tanh(c)^2)$$

$$da = [\, dc_{new} \cdot c_{prev} \cdot f(1-f), \quad dc_{new} \cdot g \cdot i(1-i), \quad dc_{new} \cdot i \cdot (1-g^2), \quad do \cdot o(1-o) \,]$$

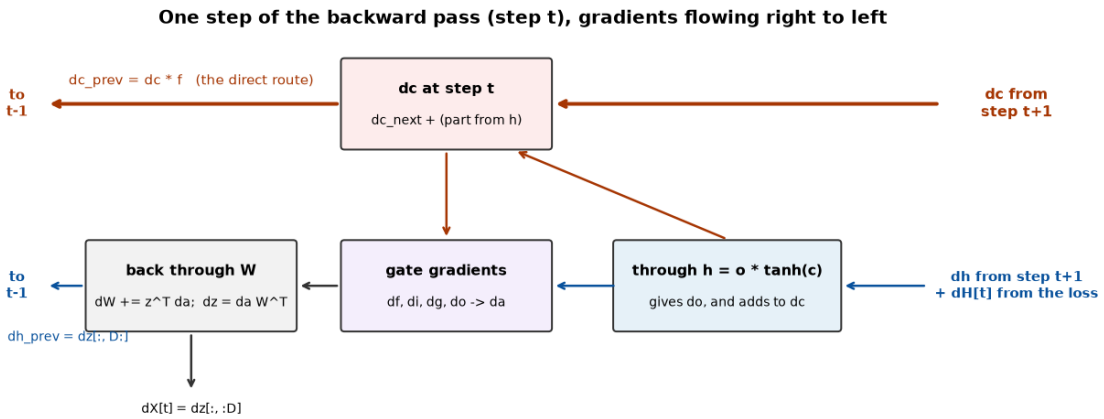
$$dW \mathrel{+}= z^T da, \quad dz = da\, W^T, \quad dc_{prev} = dc_{new} \cdot f$$

CODE

```
def lstm_backward(dH, dh_last, dc_last, cache):
    X, h0, c0, W, mask = cache["X"], cache["h0"], cache["c0"], cache["W"], cache["mask"]
    Hs, Cs, F, I, G, O = (cache[k] for k in ["H", "C", "F", "I", "G", "O"])
    m, T, D = X.shape
    dW, db, dX = np.zeros_like(W), np.zeros(W.shape[1]), np.zeros_like(X)
    dh, dc = dh_last.copy(), dc_last.copy()
    for t in reversed(range(T)):
        dh = dh + dH[:, t]
        m_t = mask[:, t:t + 1]
        h_prev = Hs[:, t - 1] if t > 0 else h0
        c_prev = Cs[:, t - 1] if t > 0 else c0
        f, i, g, o = F[:, t], I[:, t], G[:, t], O[:, t]
        tanh_c = np.tanh(Cs[:, t])
        dh_real, dc_real = dh * m_t, dc * m_t
        do = dh_real * tanh_c
        dc_new = dc_real + dh_real * o * (1.0 - tanh_c ** 2)
        da = np.concatenate([dc_new * c_prev * f * (1.0 - f),
                             dc_new * g * i * (1.0 - i),
                             dc_new * i * (1.0 - g ** 2),
                             do * o * (1.0 - o)], axis=1)
        z = np.concatenate([X[:, t], h_prev], axis=1)
        dW += z.T @ da
        db += da.sum(axis=0)
        dz = da @ W.T
        dX[:, t] = dz[:, :D]
        dh = dz[:, D:] + dh * (1.0 - m_t)
        dc = dc_new * f + dc * (1.0 - m_t)
    return dW, db, dX, dh, dc
```

- `X, h0, c0, W, mask = cache["X"], ...` — unpack several dictionary entries in one line.
- `(cache[k] for k in ["H", "C", ...])` — a generator expression unpacked into six names.
- `dh_last.copy(), dc_last.copy()` — copies, so the running dh and dc never change the caller's arrays.
- `Hs[:, t - 1] if t > 0 else h0` — the previous state, or the starting state at step 0.
- `dh * m_t` — zero for padding rows: a padded step passes no gradient into its own gates.
- `dc_real + dh_real * o * (1.0 - tanh_c ** 2)` — the cell state gets gradient from the next step and through h = o * tanh(c).
- `np.concatenate([... four gate gradients ...], axis=1)` — glued in the same f, i, g, o order as the forward pass, so they line up with W's columns.
- `f * (1.0 - f)` — the sigmoid's slope written with its own output.
- `dz = da @ W.T` — back through the matrix product: one gradient for [x, h_prev].
- `dz[:, :D], dz[:, D:]` — split it: the first D columns belong to x_t, the rest to h_prev.
- `dc_new * f + dc * (1.0 - m_t)` — the direct route on real steps; padding passes dc through unchanged.
- `return dW, db, dX, dh, dc` — after the loop, dh and dc are the gradients at h0 and c0.

SYSTEM DIAGRAM



lstm_backward's loop body as a picture: dc along the top, dh along the bottom.

Segment 2 — The classifier

LSTMClassifier

CODE

```
class LSTMClassifier:
    def __init__(self, input_dim, hidden_dim, n_classes, seed=7, forget_bias=1.0, lr=0.05,
        clip=1.0):
        self.W, self.b = init_lstm(input_dim, hidden_dim, seed, forget_bias)
        rng = np.random.RandomState(seed + 1)
        self.Why = rng.randn(hidden_dim, n_classes) / np.sqrt(hidden_dim)
        self.by = np.zeros(n_classes)
        self.H, self.lr, self.clip = hidden_dim, lr, clip
    def forward(self, X, mask=None):
        m = X.shape[0]
        zeros = np.zeros((m, self.H))
        self.Hs, self.Cs, self.cache = lstm_forward(X, zeros, zeros, self.W, self.b, mask)
        self.h_last = self.Hs[:, -1]
        self.probs = softmax(self.h_last @ self.Why + self.by)
        return self.probs
    def loss(self, y):
        return -np.mean(np.log(self.probs[np.arange(len(y)), y] + 1e-12))
    def gradients(self, y):
        m = len(y)
        dlogits = self.probs.copy()
        dlogits[np.arange(m), y] -= 1.0
        dlogits /= m
        grads = dict(Why=self.h_last.T @ dlogits, by=dlogits.sum(axis=0))
        dh_last = dlogits @ self.Why.T
        grads["W"], grads["b"], self.dX, _, _ = lstm_backward(np.zeros_like(self.Hs), dh_last,
            np.zeros_like(dh_last), self.cache)
        return grads
    def train_step(self, X, y):
        self.forward(X)
        grads, norm, _ = clip_by_global_norm(self.gradients(y), self.clip)
        for name, g in grads.items():
            getattr(self, name)[...] -= self.lr * g
    def accuracy(self, X, y):
        return (self.forward(X).argmax(axis=1) == y).mean()
```

- `rng = np.random.RandomState(seed + 1)` — a second, different seed for the output layer, so it does not reuse the LSTM's random numbers.

- `self.Hs[:, -1]` — the last column: because padding carries h forward, this is the state after each sequence's last real step.
- `dlogits[np.arange(m), y] -= 1.0` — subtract 1 at each row's true class: probabilities minus one-hot.
- `dlogits /= m` — average over the batch, in place.
- `grads["W"], grads["b"], self.dX, _, _ = lstm_backward(...)` — five return values; the gradients at h0 and c0 are not needed here.
- `np.zeros_like(self.Hs)` — no gradient enters at the hidden states except the last: a many-to-one readout.
- `grads, norm, _ = clip_by_global_norm(...)` — Day 60's clipper; the scale it applied is not needed.
- `getattr(self, name)[...] -= self.lr * g` — update each weight array in place, chosen by its name.

Segment 3 — Gradient checks

numeric_grad and report

MATH

$$slope \approx (L(w + \varepsilon) - L(w - \varepsilon)) / 2\varepsilon; \quad pass\ if\ |numeric - analytic| \le 1e\text{-}10 + 1e\text{-}6 \cdot |numeric|$$

CODE

```
def numeric_grad(f, P, eps=1e-5):
    numeric = np.zeros_like(P)
    for pos in np.ndindex(P.shape):
        old = P[pos]
        P[pos] = old + eps; up = f()
        P[pos] = old - eps; down = f()
        P[pos] = old
        numeric[pos] = (up - down) / (2 * eps)
    return numeric
def report(name, numeric, analytic):
    diff = np.abs(numeric - analytic)
    rel = diff / (np.abs(numeric) + np.abs(analytic) + 1e-12)
    ok = bool(np.all(diff <= 1e-10 + 1e-6 * np.abs(numeric)))
    print(f"{name:>7}{numeric.size:>9}{rel.max():>20.2e}{diff.max():>18.2e}{str(ok):>8}")
    return ok
```

- `def numeric_grad(f, P, eps=1e-5):` — **f** is a function with no arguments that returns the loss; P is changed in place and restored.
- `np.ndindex(P.shape)` — every index of an array of any shape.
- `P[pos] = old + eps; up = f()` — two statements on one line: nudge, then measure.
- `bool(np.all(...))` — one True/False for the whole array.
- `return ok` — report() prints a row and returns whether it passed, so the caller can combine rows with all().

The two checks

CODE

```
def loss():
    Hs, Cs, _ = lstm_forward(X, h0, c0, W, b, mask)
    return np.sum(R1 * Hs * mask[:, :, None]) + np.sum(R2 * Hs[:, -1]) + np.sum(R3 * Cs[:, -1])
_, _, cache = lstm_forward(X, h0, c0, W, b, mask)
dW, db, dX, dh0, dc0 = lstm_backward(R1 * mask[:, :, None], R2, R3, cache)
print(f"\n{'array':>7}{'entries':>9}{'max relative error':>20}{'max
    |difference|':>18}{'pass':>8}")
ok = all([report(name, numeric_grad(loss, P), A) for name, P, A in
          [("W", W, dW), ("b", b, db), ("X", X, dX), ("h0", h0, dh0), ("c0", c0, dc0)]])
print(f"every entry passes: {ok};  gradient at padded inputs exactly zero: {np.all(dX[mask == 0]
    == 0)}")
f = lambda: (net.forward(X, mask), net.loss(y))[1]
```

- `def loss():` — a function defined inside the demo; it reads X, W, b, h0, c0 from around it, so nudging them changes its result.
- `R1 * Hs * mask[:, :, None]` — weight every hidden state, ignoring padded positions.
- `all([report(...) for ... in [...]])` — a list of True/False from every row; all() is True only if every row passed.
- `dX[mask == 0]` — boolean indexing with a 2-D mask on a 3-D array: every input vector at a padded position.
- `(net.forward(X, mask), net.loss(y))[1]` — build a tuple to run forward first, then keep only the loss.

Segment 4 — demo_direct_route

The direct route as a gradient

MATH

$$dL/dc_0 = dL/dc_T \cdot \Pi_t f_t \quad \textit{when the gates do not depend on } h$$

CODE

```
for label, cut in [("gates cut off from h", True), ("normal LSTM", False)]:
    Wc = W.copy()
    if cut:
        Wc[1:, :] = 0.0
    _, _, cache = lstm_forward(X, zeros, zeros, Wc, b)
    _, _, _, _, dc0 = lstm_backward(np.zeros((m, T, H)), np.zeros((m, H)), dc_T, cache)
    direct = dc_T * np.prod(cache["F"], axis=1)
    print(f"\n{label}: ||dL/dc_0|| = {np.linalg.norm(dc0):.6e},  direct route alone =
        {np.linalg.norm(direct):.6e},  "
          f"max |difference| = {np.max(np.abs(dc0 - direct)):.2e}")
```

- `W.copy()` — a copy, so cutting connections does not change W for the second version.
- `Wc[1:, :] = 0.0` — rows 1 onward multiply h; zero them and the gates can no longer see h.
- `_, _, _, _, dc0 = lstm_backward(...)` — keep only the fifth return value.
- `np.prod(cache["F"], axis=1)` — multiply the forget gates over the time axis: one product per example and unit.

Segment 5 — demo_gradient_reach

Gradient reaching the first input

CODE

```
X, y = make_sign_task(400, T, seed=999)
row = []
for fb in [0.0, 1.0, 2.0]:
    net = LSTMClassifier(1, 16, 2, seed=7, forget_bias=fb)
    net.forward(X[:, :, None]); net.gradients(y)
    norms = np.linalg.norm(net.dX[:, :, 0], axis=0)
    table[(T, fb)] = norms[0] / norms[-1]
```

- `X[:, :, None]` — add a last axis: (400, T) becomes (400, T, 1), one input per step.
- `net.forward(...); net.gradients(y)` — two calls on one line; gradients() also stores dL/dX in net.dX.
- `np.linalg.norm(net.dX[:, :, 0], axis=0)` — the gradient's size at each position, taken over all 400 examples.
- `norms[0] / norms[-1]` — first position relative to the last.

Segment 6 — demo_training

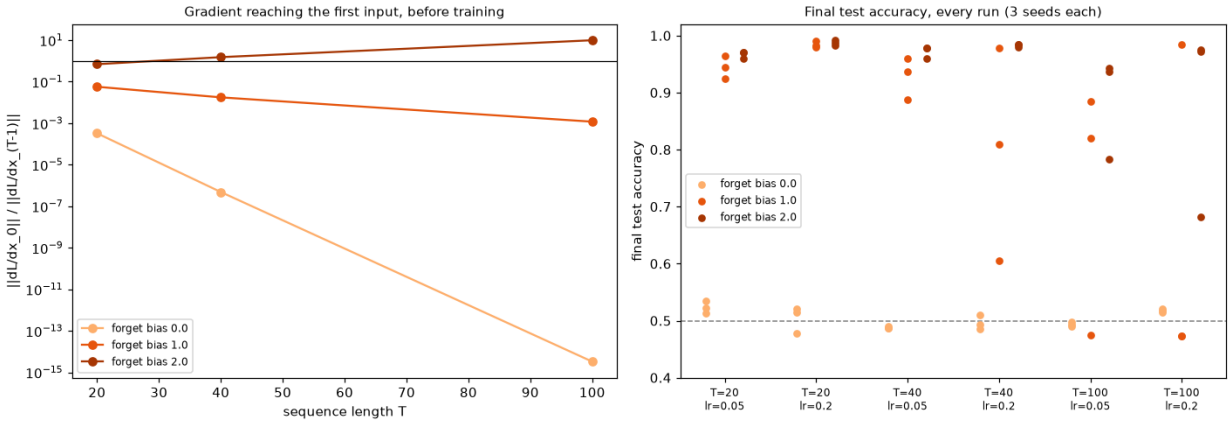
The training grid

CODE

```
results = {}
print(f"\n{'T':>4}{'f-bias':>8}{'lr':>6}   {'peak / final test accuracy, seeds 7, 8,
    9':<44}{'final >= 0.9':>13}")
for T in [20, 40, 100]:
    X_tr, y_tr = make_sign_task(600, T, seed=42)
    X_te, y_te = make_sign_task(400, T, seed=999)
    X_tr, X_te = X_tr[:, :, None], X_te[:, :, None]
    for fb in [0.0, 1.0, 2.0]:
        for lr in [0.05, 0.2]:
            runs = []
            for seed in [7, 8, 9]:
                net = LSTMClassifier(1, 16, 2, seed=seed, forget_bias=fb, lr=lr, clip=1.0)
                rng = np.random.RandomState(seed)
                curve = []
                for epoch in range(150):
                    order = rng.permutation(600)
                    for start in range(0, 600, 30):
                        idx = order[start:start + 30]
                        net.train_step(X_tr[idx], y_tr[idx])
                    if epoch % 10 == 0 or epoch == 149:
                        curve.append(net.accuracy(X_te, y_te))
                runs.append((max(curve), curve[-1], curve))
            results[(T, fb, lr)] = runs
            cells = "  ".join(f"{p:.3f}/{f:.3f}" for p, f, _ in runs)
            print(f"{T:>4}{fb:>8}{lr:>6}   {cells:<44}{sum(f >= 0.9 for _, f, _ in runs):>9} of
                3")
```

- `for T ... for fb ... for lr ... for seed ...` — four nested loops: 3 × 3 × 2 × 3 = 54 runs.
- `rng.permutation(600)` — a new random order of the training sequences each epoch.
- `if epoch % 10 == 0 or epoch == 149:` — evaluate every 10 epochs and at the very end.
- `runs.append((max(curve), curve[-1], curve))` — peak, final and the whole curve for one seed.
- `" ".join(f"{p:.3f}/{f:.3f}" for p, f, _ in runs)` — one peak/final pair per seed, joined with two spaces.
- `sum(f >= 0.9 for _, f, _ in runs)` — True counts as 1, so this counts the seeds that learned.

REAL GRAPH



Real graph from the script: the gradient reaching x_0 before training, and the final accuracy of all 54 runs.

Segment 7 — main

main

CODE

```
def main():
    with threadpool_limits(limits=1, user_api="blas"):
        demo_gradient_checks()
        demo_direct_route()
        reach = demo_gradient_reach()
        results = demo_training()
        plot_results(reach, results)
        draw_backward_diagram()
        note_on_what_comes_next()
```

- `with threadpool_limits(limits=1, user_api="blas"):` — everything inside runs with the matrix library on one thread; it is restored when the block ends.
- `reach = demo_gradient_reach()` — functions return what later steps need; plot_results receives both tables.

Quick Syntax Glossary — new or reinforced constructs

- `a, b = d["a"], d["b"]` — unpack several dictionary entries in one line.
- `x.copy()` — an independent copy; changing it leaves the original alone.
- `dh * m_t` — zero out padded rows with a 0/1 mask.
- `s * (1 - s)` — the sigmoid's slope from its output s.
- `np.concatenate([...], axis=1)` — glue blocks side by side, in the order W expects.
- `dz[:, :D], dz[:, D:]` — split columns between the input part and the hidden part.
- `np.prod(A, axis=1)` — multiply along one axis.
- `def f(): ... (a closure)` — a function that reads variables from where it was defined.
- `all([...])` — True only if every item is True.
- `sum(x >= 0.9 for x in xs)` — count how many items meet a condition.
- `with threadpool_limits(...):` — a temporary setting that undoes itself at the end of the block.

End of Day 65 syntax reference. For the experiments and what they showed, see the Day 65 complete guide.